



POLITECNICO
MILANO 1863

DEPARTMENT OF ELECTRONICS
INFORMATION AND BIOENGINEERING

Project Assignment - Updates & Details

Embedded and Edge Artificial Intelligence

08.05.2026 | Hazem Hesham Yousef Shalby

Updates from the Previous Presentation

- Tensor arena (SRAM) limit updated: the constraint checker enforces **300 KB** (not 400 KB as shown in the previous slides). The Flash budget remains < 800 KB.
- The performance evaluation script has been finalized. It computes **F1 detection score** using Hungarian matching at 1.0 m threshold.
- The constraint checker has been updated: it now correctly estimates only the model activation memory (SRAM), not the full model size. It also checks for **forbidden operation types** beyond LSTM/GRU/RNN, including CUSTOM and Flex ops that are incompatible with TensorFlow Lite for Microcontrollers.
- The repository template is now available on **GitHub Classroom** (see last section of this presentation).

ESP32-S3 Deployment Constraints (Updated)

Check	Limit	Notes
Model file size	< 800 KB	Flash budget
Activation arena	< 300 KB	SRAM budget - model only
Quantization	INT8 recommended	4× smaller, faster on ESP32-S3
Forbidden ops	No LSTM/GRU/RNN/CUSTOM/Flex	Not supported by TFLM

Note: The 300 KB limit applies to the model's tensor arena only. Your preprocessing and postprocessing code in code.py also consumes SRAM. Design accordingly.

Two Scripts, Two Purposes

evaluate_performance.py

- Measures how well your model localises people
- Run it locally before every submission

Command:

```
python evaluation/evaluate_performance.py \  
  --gt-path evaluation/example/output_test.jsonl \  
  --pred-path evaluation/example/my_output.jsonl
```

evaluate_constraint.py

- Checks whether your model fits on ESP32-S3
- Hard failure = submission rejected

Command:

```
python evaluation/evaluate_constraint.py \  
  --model-path submission/model.tflite
```

Performance Evaluation - F1 Score

Step 1 - Format Validation

Your .jsonl file is validated before any score is computed:

- Valid JSON, no duplicate frames
- Max 4 localisations per frame
- All coordinates within room bounds $[0, 4.8] \times [0, 7.2]$ m
- Any error $\rightarrow$ script exits, no score produced

Step 2 - Hungarian Matching per Frame

For each frame, predicted positions are matched to ground-truth persons using optimal assignment (Hungarian algorithm).

- A match is a **True Positive** only if distance ≤ 1.0 m
- Unmatched predictions $\rightarrow$ **False Positives**
- Unmatched GT persons $\rightarrow$ **False Negatives**

Performance Evaluation — F1 Score (cont.)

Step 3 - Global F1 Aggregation

Metrics are computed globally across all frames:

- $\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$
- $\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$
- $\text{F1} = 2 \times \text{P} \times \text{R} / (\text{P} + \text{R})$

Step 4 - Additional Metrics (informational only)

Reported on matched pairs only:

- RMSE, MAE, median error, P90
- Count MAE, count accuracy (person count per frame)

Note: frames emitted by your code but not present in the ground truth are ignored. Ground-truth frames with no prediction are treated as all False Negatives.

Constraint Checker - 5 Checks

CHECK 1 - TFLite Validity

Model must load and allocate tensors successfully. Failure here stops all further checks.

CHECK 2 - File Size

model.tflite must be < 800 KB. Hard failure if exceeded.

CHECK 3 - Activation Arena (SRAM)

Estimated by running a dummy inference and measuring which tensors change - those are activations in SRAM. Weight tensors (Flash) are not counted. Limit: < 300 KB. Hard failure if exceeded.

Constraint Checker - 5 Checks (cont.)

CHECK 4 - Forbidden Operations

Scans the model's op list. Flags:

- **LSTM, GRU, RNN variants** (recurrent layers not supported by TFLM)
- **CUSTOM ops, Flex ops** (require full TF runtime, unavailable on ESP32-S3)

Hard failure if any detected.

CHECK 5 - Quantization (advisory)

- Full INT8 → PASS
- Hybrid quantization → WARNING
- float32 → WARNING (4× larger, slower)

This check is advisory only and does not cause a hard failure.

Exit codes: 0 = all hard constraints pass | 1 = at least one failed

What You Will Find in Your GitHub Classroom Repository

```
submission/  
  code.py  ← your inference script (do not rename)  
  model.tflite ← your trained TFLite model (do not rename)  
report/  
  report.pdf ← your project report  
src/  
  ... ← training notebooks, scripts, experiments  
evaluation/ ← provided, do not modify  
  evaluate_constraint.py  
  evaluate_performance.py  
example/  
  input_test.npy  
  output_test.jsonl  
requirements.txt ← allowed Python libraries for code.py
```

Important: The evaluation/ folder and requirements.txt are protected. You cannot and must not modify their contents. Run the scripts locally to verify your submission before pushing.

Allowed Python Libraries - requirements.txt

The file **requirements.txt** in the root of your repository lists the **only third-party libraries** allowed in **code.py**.

- **numpy** $\geq$ **2.0.0**
- **scipy** $\geq$ **1.10.0**
- **tensorflow** $\geq$ **2.13.0**

During evaluation, a **clean environment** will be created with **only these libraries installed**. Any import not satisfied by this environment will cause your submission to fail.

Python standard library modules (argparse, json, sys, pathlib, math, ...) are always available.

Always Test Locally Before Pushing

Step 1 - Run your inference script

```
python submission/code.py \  
  --input-path  evaluation/example/input_test.npy \  
  --output-path evaluation/example/my_output.jsonl
```

Step 2 - Check performance

```
python evaluation/evaluate_performance.py \  
  --gt-path    evaluation/example/output_test.jsonl \  
  --pred-path  evaluation/example/my_output.jsonl
```

Step 3 - Check ESP32-S3 constraints

```
python evaluation/evaluate_constraint.py \  
  --model-path submission/model.tflite
```

⚠ Warning: The example input/output is derived from training data and is provided for format verification only, not for performance benchmarking. The final evaluation uses a separate acquisition session not seen during training.

Accessing Your Repository - Step by Step

Step 1 - Accept the Assignment

Click the invitation link: <https://classroom.github.com/a/8dqcBAR3>

You will be redirected to GitHub Classroom.

- You need a GitHub account - if you don't have one, create it before clicking the link
- Authorise GitHub Classroom to access your account when prompted

Step 2 - Create or Join a Group

You must form groups of **exactly 2 students**.

- Either create a new group and share the name with your partner
- Or search for your partner's existing group and join it
- The system allows a maximum of 2 members per group

Accessing Your Repository - Step by Step (cont.)

Step 3 - Group Naming Convention

Groups must be named with the format:

<person_code_1>-<person_code_2>

Example: 10xxxxxx-10yyyyyy

- Use the **person code**
- Separate both codes with a hyphen (-), no spaces

Step 4 - Repository Is Ready

Once both members have joined, your shared repository is created automatically.

- The template with all required folders is already in place
- Evaluation scripts are pre-installed in the repo
- Clone the repo and start working immediately

Group Formation Rules - Read Carefully

- Groups must consist of **exactly 2 students**. No exceptions.
- Once a group is created, the system allows a maximum of 2 members.

⚠ IMPORTANT - Unauthorized Group Members

The system technically allows a person to join any open group. However:

- If an unauthorized person joins a group, this constitutes a **violation of the Politecnico di Milano academic integrity code**.
- The project submission will be **declared invalid** for the unauthorized.
- An invalid project directly results in the student being **referred to the next academic year**, as the project is a mandatory component of the final grade.

Key Takeaways

- Constraint limits: model < **800 KB Flash**, activation arena < **300 KB SRAM**, no LSTM / GRU / RNN / CUSTOM / Flex ops.
- Run **both evaluation scripts** locally before every push.
- The final evaluation uses unseen data - build a model that generalises.
- Groups of exactly 2 students, named <person_code_1>-<person_code_2>.
- **Unauthorized group members = project invalidation = academic penalty.**
- Deadlines: **July 15** (June/July session) | **September 1** (September session).